Chapter 11 — Agents

11.1 What an Agent Is, and When Not to Build One

Conceptual Deep Dive

An **agent** is an LLM in a loop that decides actions, calls tools, observes results, and continues until a goal is met or a limit hits. Spectrum of autonomy: single call → chain/workflow (fixed steps) → router → agent (LLM chooses steps) → multi-agent. Prefer the lowest autonomy that solves the task: workflows are cheaper, faster, testable and predictable. Build agents when the number/order of steps is unknowable up front (research, coding, open-ended support). Core loop: *perceive (context)* → *plan* → *act (tool)* → *observe* → *update state* → *stop?* Key reliability levers: tight tool set, clear stop criteria, step/cost/time budgets, deterministic scaffolding around the LLM, and human checkpoints on irreversible actions. Errors compound: 95% per-step reliability over 10 steps ≈ 60% end-to-end.

Cross-Connection Scenarios

Tool schemas (7), ReAct (6.3), eval trajectories (10), guardrails (14), idempotent side effects (4.5).

Production Anti-Patterns

- Using an agent where a 3-step workflow suffices.
- No budget/timeouts; runaway loops.
- Agents granted broad credentials.

Minimal Working Code

```
import json
import time

def agent_loop(llm, tools: dict, goal: str, max_steps=8, budget_s=60):
    messages = [{"role": "user", "content": goal}]
    t0 = time.time()
    for step in range(max_steps):
        if time.time() - t0 > budget_s:
            return "budget exceeded"
        reply = llm(messages)                    # returns {"content":..., "tool_call"
:{name,args}|None}
        messages.append({"role": "assistant", "content": reply["conten
t"]})
        call = reply.get("tool_call")
        if not call:
            return reply["content"]              # model decided it is done
        try:
            result = tools[call["name"]](**call["args"])
        except Exception as e:
            result = f"ERROR: {e}"
        messages.append({"role": "tool", "content": json.dumps(result, defa
ult=str)[:4000]})
    return "max steps reached"
```

11.2 Planning Patterns

Conceptual Deep Dive

- **ReAct**: interleaved reasoning and acting; flexible, greedy.
- **Plan-and-Execute**: planner writes a full plan; executor runs steps; a replanner revises on failure. Cheaper (small model executes) and more inspectable.
- **ReWOO**: plan with variable placeholders, execute tools in parallel, then solve.
- **Reflexion / self-critique**: after failure, write a verbal lesson into memory and retry.
- **Tree/graph search** (LATS, ToT): explore alternatives with scoring.
- **Hierarchical**: manager decomposes, workers handle sub-tasks.
- **Verifier loops**: separate checker validates outputs (tests for code, schema for data) before accepting. Prefer verifiable feedback (unit tests, compilers, SQL execution) over LLM opinions.

Cross-Connection Scenarios

Reasoning prompts (6.2); evals of plan quality (10); cost control by routing executor to small models (2.2).

Production Anti-Patterns

- Plans that never get revised when step 2 invalidates step 5.
- Planner and executor with different tool knowledge.
- Self-critique without any external signal (confidently wrong loops).

Minimal Working Code

```
def plan_and_execute(planner, executor, replanner, goal, max_replans=2
):
    plan = planner(goal)                    # list[str]
    done, replans = [], 0
    while plan:
        step = plan.pop(0)
        ok, out = executor(step, done)
        if ok:
            done.append((step, out))
            continue
        if replans >= max_replans:
            return {"status": "failed", "done": done, "failed_step&quo
t;: step}
        replans += 1
        plan = replanner(goal, done, step, out)
    return {"status": "ok", "done": done}
```

11.3 State Management and Orchestration with LangGraph

Conceptual Deep Dive

Model an agent as a **graph**: nodes are functions (LLM call, tool, human review), edges are transitions (conditional routing), and a typed **state** object flows through, updated by **reducers** (e.g. append to messages). **LangGraph** adds persistence via **checkpointers** (resume after crash, time-travel debugging), **human-in-the-loop interrupts** (pause for approval), streaming, subgraphs, and a Send API for map-reduce parallelism. Benefits over a while loop: explicit control flow, durable execution, observability per node, and testability of each node in isolation. Alternatives: Temporal/Durable Functions for workflow durability; PydanticAI, OpenAI Agents SDK, Claude Agent SDK, CrewAI, LlamaIndex Workflows, Semantic Kernel, Haystack.

Cross-Connection Scenarios

Checkpoint stores use Postgres/Redis (13); traces per node (14); interrupts are guardrails (14).

Production Anti-Patterns

- Storing everything in global variables rather than typed state.
- No checkpointing → lost work on pod restart.

- Unbounded message-list growth in state.

Minimal Working Code

```
# pip install langgraph
from typing import Annotated, TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages

class State(TypedDict):
    messages: Annotated[list, add_messages]
    attempts: int

def draft(state: State):
    return {"messages": [("assistant", "draft answer")], "attempts&quo
t;: state["attempts"] + 1}

def check(state: State):
    return "done" if state["attempts"] >= 2 else "retry"

g = StateGraph(State)
g.add_node("draft", draft)
g.add_edge(START, "draft")
g.add_conditional_edges("draft", check, {"done": END, "retry": "draft&
quot;})
app = g.compile()
print(app.invoke({"messages": [("user", "hi")], "attempts": 0})[&
quot;attempts"])
```

11.4 Memory Architectures

Conceptual Deep Dive

- **Working memory (short-term)**: the context window / conversation buffer. Manage with sliding windows, summarisation of old turns, and pruning of stale tool outputs.
- **Episodic memory**: records of past interactions/events ("last Tuesday the user asked about X"), stored with timestamps and retrieved by recency + similarity.
- **Semantic memory**: distilled facts and preferences ("user prefers metric units"), stored as key–value or knowledge-graph entries; updated by an LLM "memory manager" that adds, edits or deletes facts.
- **Procedural memory**: learned instructions/skills; e.g. the system prompt or tool recipes refined over time. Retrieval scoring (Generative Agents): $\alpha \cdot \text{recency} + \beta \cdot \text{importance} + \gamma \cdot \text{relevance}$. Frameworks: Letta (MemGPT, OS-style paging between core and archival memory), mem0, Zep, LangGraph store. Concerns: privacy and deletion rights (GDPR), memory poisoning via injected "facts", and staleness/conflict resolution.

Cross-Connection Scenarios

Vector search (8), RAG (9), privacy and PII (14), context engineering (6.4).

Production Anti-Patterns

- Appending every turn forever.
- Writing LLM-inferred "facts" without confidence or provenance.
- No user-visible way to view/delete memories.

Minimal Working Code

```
import math
import time

class Memory:
```

```
    def __init__(self, embed):
        self.embed, self.items = embed, []

    def add(self, text: str, importance: float = 0.5, kind: str = "episodic"):
        self.items.append({"text": text, "v": self.embed(text), "t": time.t
ime(),
                           "imp": importance, "kind": kind})

    def recall(self, query: str, k=3, half_life_s=86400):
        q = self.embed(query)
        def score(m):
            sim = sum(a * b for a, b in zip(q, m["v"]))
            recency = 0.5 ** ((time.time() - m["t"]) / half_life_s)
            return sim + 0.5 * recency + 0.5 * m["imp"]
        return sorted(self.items, key=score, reverse=True)[:k]

def summarize_if_long(llm, history: list[str], max_items=20):
    if len(history) <= max_items:
        return history
    old, recent = history[:-10], history[-10:]
    return [f"Summary: {llm('Summarize: ' + chr(10).join(old))}"] + recent
```

11.5 Multi-Agent Coordination: AutoGen, CrewAI and Patterns

Conceptual Deep Dive

Patterns: **supervisor/worker** (router agent delegates), **swarm/handoff** (agents transfer control to peers), **debate/critic** (propose + critique), **pipeline** (specialists in sequence), **blackboard** (shared state). **AutoGen** (and its successor Microsoft Agent Framework) models conversational agents exchanging messages in group chats with speaker-selection; **CrewAI** assigns role-based agents tasks in a crew; LangGraph expresses these as explicit graphs. Multi-agent helps with context isolation (each agent sees a small context), parallelism, and specialisation — but adds cost, latency, and failure surface. Keep communication structured (typed messages, not free chat), set turn limits, and give one agent final authority.

Cross-Connection Scenarios

Protocols: MCP for tools and A2A for agent-to-agent interoperability (7.4); eval of end-to-end trajectories (10); cost control (2.4).

Production Anti-Patterns

- "Agents chatting" with no termination condition.
- Using five agents where one with tools works.
- Shared mutable state without ownership.

Minimal Working Code

```
def supervisor(llm, workers: dict, task: str, max_rounds=6):
    log, results = [], {}
    for _ in range(max_rounds):
        decision = llm(f"Task: {task}\nResults so far: {results}\n"
                       f"Workers: {list(workers)}\nReply 'DONE: <answer>' or '<worker>:
<instruction>'")
        if decision.startswith("DONE:"):
            return decision[5:].strip()
        name, _, instruction = decision.partition(":")
        if name.strip() not in workers:
            results["error"] = f"unknown worker {name}"
            continue
        results[name.strip()] = workers[name.strip()](instruction.strip())
    return "round limit reached"
```

11.6 Evaluating and Securing Agents

Conceptual Deep Dive

Evaluate **trajectories**, not just final answers: tool-selection accuracy, argument correctness, redundant steps, recovery after errors, cost per solved task, and success rate over N repeated runs (**pass^k** consistency). Benchmarks: SWE-bench, τ-bench, WebArena, GAIA — use for orientation, then build task-specific simulators. Security: least privilege, per-action authorisation, sandboxed execution (containers/gVisor/Firecracker), network egress allow-lists, secret isolation, human approval for destructive actions, and audit logs.

Cross-Connection Scenarios

Eval (10), injection (6.5, 14), observability traces (14).

Production Anti-Patterns

- Evaluating on one run per case.
- Code-execution tools running on the host.
- Missing kill switch.

Minimal Working Code

```
import subprocess
import sys
import tempfile

def run_sandboxed(code: str, timeout=5) -> str:
    """Demo only. In production use containers/gVisor with no network and resource limits.
"""
    with tempfile.NamedTemporaryFile("w", suffix=".py", delete=False) as f:
        f.write(code)
    p = subprocess.run([sys.executable, "-I", f.name], capture_output=True, text=True, timeout=
timeout)
    return (p.stdout + p.stderr)[:2000]

def pass_hat_k(results: list[list[bool]]) -> float:
    """Fraction of tasks solved in ALL k attempts."""
    return sum(all(r) for r in results) / len(results)

print(pass_hat_k([[True, True], [True, False]]))
```

11.7 Context Management for Long-Running Agents

Conceptual Deep Dive

Agents accumulate tool outputs that fill the window. Techniques: **truncate and reference** (store large outputs externally, keep a short summary plus a handle the agent can re-read), **tool-result clearing** (drop old observations once consumed), **rolling summarisation** of earlier turns, **structured scratchpad/notes** the agent writes and re-reads (a file or state field), and **sub-agents** that do noisy exploration in a separate context and return only a distilled result. Prefer deterministic compaction rules over asking the model what to forget. Edge cases: summaries that drop a constraint the user stated early (pin hard requirements outside the summarised region), compaction invalidating prompt caching, and cyclic loops where the agent repeats an action because the earlier failure was trimmed.

Cross-Connection Scenarios

Context engineering (6.4), working memory (11.4), cost control (2.4), prompt caching (2.4).

Production Anti-Patterns

- Keeping every raw tool output forever.
- Summarising away the user's original goal and constraints.
- No loop detection.

Minimal Working Code

```
import hashlib

class ContextManager:
    def __init__(self, pinned: str, max_chars: int = 12_000, keep_last: int = 6):
        self.pinned, self.max, self.keep = pinned, max_chars, keep_last
        self.items: list[dict] = []
        self.blobs: dict[str, str] = {}

    def add_tool_result(self, name: str, text: str, inline_limit: int = 800):
        if len(text) > inline_limit:
            h = hashlib.sha1(text.encode()).hexdigest()[:8]
            self.blobs[h] = text
            text = f"{text[:inline_limit // 2]}... [truncated; full output id={h}]"
        self.items.append({"role": "tool", "name": name, "content"
;: text})

    def render(self) -> list[dict]:
        msgs = [{"role": "system", "content": self.pinned}] + self.items
        while sum(len(m["content"]) for m in msgs) > self.max and len(msgs) > self.keep +
 1:
            msgs.pop(1)                    # drop oldest non-pinned item
        return msgs

cm = ContextManager("Goal: fix bug #12. Never edit tests.")
cm.add_tool_result("grep", "x" * 5000)
print(len(cm.render()), list(cm.blobs))
```

```
def detect_loop(actions: list[tuple[str, str]], window: int = 6, repea
ts: int = 3) -> bool:
    """actions: [(tool_name, args_repr)]. True if the same call recurs too often recently.
"""
    recent = actions[-window:]
    return any(recent.count(a) >= repeats for a in set(recent))

print(detect_loop([("search", "q1")] * 3 + [("open", "u")]))
```

11.8 Durable Execution, Human-in-the-Loop and Error Recovery

Conceptual Deep Dive

Long agent runs outlive processes. **Durable execution** persists state after each step (LangGraph checkpointers, Temporal workflows) so a crash resumes instead of restarting and re-paying. Design steps to be **idempotent** or record completed side effects. **Human-in-the-loop**: pause at an interrupt with a serialisable payload (proposed action), accept approve/edit/reject, then resume. **Error recovery** ladder: retry transient errors with backoff, feed deterministic tool errors back to the model once or twice, replan, fall back to a simpler path, then escalate to a human with the full trace. Edge cases: approvals that expire, state schema changes between deploy versions (resume of old checkpoints), concurrent runs on the same thread, and timeouts while waiting for humans (days).

Cross-Connection Scenarios

Idempotency (4.5), LangGraph (11.3), approvals (7.7), observability (14.1).

Production Anti-Patterns

- Keeping agent state only in process memory.

- Resuming from a checkpoint and re-running a side effect that already happened.
- Infinite retry on a deterministic tool error.

Minimal Working Code

```
import json
import os

class RunState:
    def __init__(self, path: str):
        self.path = path
        self.s = json.load(open(path)) if os.path.exists(path) else {"step": 0, "done"
:: {}}

    def save(self):
        tmp = self.path + ".tmp"
        json.dump(self.s, open(tmp, "w"))
        os.replace(tmp, self.path)          # atomic

def run_steps(steps, state: RunState):
    for i in range(state.s["step"], len(steps)):
        name, fn = steps[i]
        if name not in state.s["done"]:
            state.s["done"][name] = fn()      # side effect recorded before advancing
        state.s["step"] = i + 1
        state.save()
    return state.s["done"]
```

```
def recover(step_fn, replan_fn, escalate_fn, transient=(TimeoutError,)
, max_retries=2):
    for attempt in range(max_retries + 1):
        try:
            return step_fn()
        except transient:
            continue
        except ValueError as e:                # deterministic error: give the model one chance
            alt = replan_fn(str(e))
            if alt is not None:
                return alt()
            break
    return escalate_fn()
```

11.9 Coding, Browser and Computer-Use Agents

Conceptual Deep Dive

Coding agents read a repository, plan edits, modify files, run tests and iterate. Effective designs give the agent: a repo map or search tools (grep, symbol lookup) rather than whole-repo dumps; precise edit tools (apply a patch, replace a unique string) instead of rewriting files; a test runner with trimmed output; and a sandbox with no production credentials. Verification via tests and linters is the strongest feedback loop. **Browser agents** observe pages through the accessibility tree or DOM text (cheaper and more reliable than raw screenshots), act with click/type/navigate primitives, and need guards against prompt injection from page content and irreversible actions (purchases, deletions). **Computer-use agents** operate GUIs via screenshots and coordinates; slower and less reliable, so prefer APIs, then structured DOM access, then pixels. Common controls: step budgets, per-domain allow-lists, human approval for sensitive actions, session recording for audit. Edge cases: CAPTCHAs and bot detection, dynamic pages and timing, stale element references, huge diffs, flaky tests mistaken for regressions, and agents "fixing" tests by weakening them.

Cross-Connection Scenarios

Tool safety (7.7), sandboxing (11.6), injection (6.5, 14.2), trajectory evaluation (11.6, 17.4), long context management (11.7).

Production Anti-Patterns

- Letting the agent edit the tests it is judged by.
- Giving a browser agent logged-in sessions with payment access.
- Feeding full HTML pages (tens of thousands of tokens) as observations.

Minimal Working Code

```
import re
from pathlib import Path

def apply_unique_replace(path: str, old: str, new: str) -> str:
    """Edit tool: refuse ambiguous edits so the agent must supply enough context."&qu
ot;"
    p = Path(path)
    text = p.read_text(encoding="utf-8")
    n = text.count(old)
    if n == 0:
        return "ERROR: old string not found"
    if n > 1:
        return f"ERROR: old string appears {n} times; include more surrounding context"
    p.write_text(text.replace(old, new), encoding="utf-8")
    return "OK"

def trim_test_output(out: str, max_lines: int = 40) -> str:
    lines = out.splitlines()
    fails = [l for l in lines if re.search(r"(FAILED|ERROR|AssertionError|Traceback)", l)]
    return "\n".join((fails + lines[-10:])[:max_lines])
```

```
def protected_paths_check(changed_files: list[str], protected: list[st
r]) -> list[str]:
    """Reject runs that modify tests/CI config the agent is evaluated on."""
;
    import fnmatch
    return [f for f in changed_files if any(fnmatch.fnmatch(f, pat) for pat in protected)]

print(protected_paths_check(["src/app.py", "tests/test_app.py"], ["tests/*"
, ".github/*"]))
```

11.10 Agent Interoperability: MCP, A2A and Tool Registries

Conceptual Deep Dive

Two protocol layers are emerging. **Tool/context protocols** (MCP) connect an agent to tools and data. **Agent-to-agent protocols** (A2A and similar) let independently built agents discover each other through **agent cards** (capabilities, endpoints, auth), delegate **tasks** with lifecycle states (submitted, working, input-required, completed, failed), and exchange artifacts. Design questions: identity and authorisation across organisational boundaries, task timeouts and cancellation, streaming progress, schema negotiation, and trust (an external agent's output is untrusted data). A **tool registry** with metadata (owner, risk tier, rate limits, schema version, cost) enables governance, discovery and per-tenant enablement; retrieve tools dynamically by embedding descriptions when the catalogue is large. Edge cases: capability claims that overstate reality, loops where agents delegate to each other indefinitely, and version skew between agent cards and behaviour.

Cross-Connection Scenarios

MCP (7.8), multi-agent patterns (11.5), tool retrieval (7.5), governance (14.7).

Production Anti-Patterns

- Trusting a remote agent's claimed capabilities without contract tests.
- No delegation depth limit.
- Passing the user's full context to every delegate.

Minimal Working Code

```
from dataclasses import dataclass, field

@dataclass
class AgentCard:
    name: str
    url: str
    skills: list[str]
    auth: str = "bearer"
    version: str = "1.0"

@dataclass
class Task:
    id: str
    skill: str
    payload: dict
    depth: int = 0
    state: str = "submitted"
    history: list = field(default_factory=list)

def delegate(task: Task, registry: dict[str, AgentCard], call, max_depth: int = 3):
    if task.depth >= max_depth:
        task.state = "failed"
        task.history.append("delegation depth exceeded")
        return task
    card = next((c for c in registry.values() if task.skill in c.skills), None)
    if card is None:
        task.state = "failed"
        return task
    task.state = "working"
    result = call(card, Task(task.id, task.skill, task.payload, task.depth + 1))
    task.state = result.state
    return task
```

```
import numpy as np

def select_tools(query_vec: np.ndarray, tool_vecs: dict[str, np.ndarray], k: int = 5, risk_ok=lambda n: T
rue):
    """Retrieve the top-k tools by description similarity, then filter by policy."&qu
ot;"
    scored = sorted(tool_vecs, key=lambda n: -float(query_vec @ tool_vecs[n]))
    return [n for n in scored if risk_ok(n)][:k]

rng = np.random.default_rng(0)
tools = {"search": rng.standard_normal(8), "send_email": rng.standard_normal(8), &quo
t;calc": rng.standard_normal(8)}
print(select_tools(tools["calc"], tools, k=2, risk_ok=lambda n: n != "send_email"))
```

11.11 Cost, Latency and Reliability Engineering for Agents

Conceptual Deep Dive

Agent cost grows with steps x context size. Levers: **route steps to the cheapest adequate model** (planner on a strong model, executor/summariser on small ones), **cache** tool results and LLM calls (idempotent reads), **parallelise** independent tool calls, **trim context** (7.7/11.7), **early-exit** when a verifier passes, cap steps and tokens, and prefer deterministic code for steps that need no reasoning. Latency: stream intermediate progress, run speculative branches, precompute likely retrievals. Reliability: per-step timeouts, retry transient errors, circuit breakers on flaky tools, validation of every tool result, and fallbacks to simpler workflows. Measure **cost per solved task**, **steps per task**, **tool error rate**, and **p95 end-to-end time**. Edge cases: runaway loops, cost spikes from a single long document, rate-limit cascades when many agents share a key, and non-idempotent tools being retried.

Cross-Connection Scenarios

Token economics (2.4), gateways (13.6), reliability patterns (4.3–4.5), cost governance (15.7), loop detection (11.7).

Production Anti-Patterns

- Using the strongest model for every step.
- No per-task spend ceiling.
- Retrying a tool call that has side effects without an idempotency key.

Minimal Working Code

```
import time

class Budget:
    def __init__(self, max_usd: float, max_steps: int, max_seconds: float):
        self.max_usd, self.max_steps, self.max_seconds = max_usd, max_steps, max_seconds
        self.usd, self.steps, self.t0 = 0.0, 0, time.time()

    def charge(self, usd: float):
        self.usd += usd
        self.steps += 1

    def exhausted(self) -> str | None:
        if self.usd > self.max_usd:
            return "cost"
        if self.steps >= self.max_steps:
            return "steps"
        if time.time() - self.t0 > self.max_seconds:
            return "time"
        return None

b = Budget(0.50, 20, 120)
b.charge(0.6)
print(b.exhausted())
```

```
import functools
import json

def memoize_tool(fn):
    cache = {}
    @functools.wraps(fn)
    def wrapper(**kwargs):
        key = json.dumps(kwargs, sort_keys=True, default=str)
        if key not in cache:
            cache[key] = fn(**kwargs)
        return cache[key]
    return wrapper

@memoize_tool
def lookup(customer_id: str) -> dict:
    return {"id": customer_id, "tier": "gold"}

print(lookup(customer_id="c1") is lookup(customer_id="c1"))
```